

Banglish Hate Speech XAI: Project Documentation

We built a system that detects hate speech in Bangla written in English letters (Banglish), names which of 8 target groups it attacks, and shows the words behind every decision. It reaches binary macro-F1 76.23 on the official BanTH test set, on par with the published baselines, and its explanations were checked against words marked by two people.

1. Problem, goals and users

Most Bangladeshi social media comments are typed in Banglish: Bangla words in English letters, often mixed with English. Existing Bangla hate speech tools are trained on Bangla script, so they miss this text, and the ones that do handle Banglish give a label with no reason.

Why Banglish is hard

- No fixed spelling: the same word appears as *bhai*, *vai* or *bai*, so a model cannot rely on exact words.
- Code-mixing: Bangla and English words sit in the same sentence.
- Short, informal text full of emoji and slang (the median comment has 7 words).
- Hate is multi-label: one comment can attack a political group and insult a person at the same time.

Goals

1. Detect whether a comment is hate speech (binary).
2. Name which of 8 categories apply: Political, Religious, Gender, Personal Offense, Abusive/Violence, Origin, Body Shaming, Misc (multi-label).
3. Show which words led to each decision (LIME and SHAP).
4. Check those explanations against words marked by real people, not just trust them.
5. Ship it as a working web app, not only a notebook.

Who uses it

User	What they need from the system
Content moderators	A fast first flag plus highlighted words, so they can check a comment before acting

User	What they need from the system
NGOs and monitoring groups	Which groups are targeted (categories), to track online harm
Researchers	A reproducible baseline and a small rationale dataset for transliterated Bangla
Our course (CSE 4889)	A complete ML pipeline: data, model, evaluation, explanation, demo, report

Research gap we address: no earlier work gives rationale-checked, faithfulness-tested explanations for multi-label hate speech in transliterated Bangla. Explainable hate speech exists for English (HateXplain) and Portuguese (HateBRXplain), so our claim is only about this language setting.

2. Scope: what we built and what we did not

With the deadline on 30 September 2026 we followed a 7-step sprint and moved five planned items to future work on purpose. Two extra studies (keyword bias test and bias fix) were added during the project because the explanations revealed a problem.

Item	Status	Notes
Drive folder setup, environment record, asset list	Done	Notebook 00
Load, check and clean BanTH (official splits)	Done	Notebook 01
Baseline table from the BanTH paper (cited, not re-run)	Done	Notebook 03, numbers checked against the paper
Dual-head XLM-R model (binary + 8 categories)	Done	Notebook 04, 18.4 min on a T4 GPU
LIME and SHAP explanations	Done	Notebook 05
Mini rationale set (100 comments, 2 annotators) + Kappa	Done	Notebooks 06 and 07; A did 93, B did 57, 55 overlap
Plausibility and faithfulness evaluation	Done	Notebook 07
Gradio demo app with public link	Done	Notebook 08

Item	Status	Notes
Keyword bias test (100 harmless sentences)	Done (added)	Notebook 09
Bias fix by oversampling	Done (added)	Notebook 10, reduces bias but costs category F1
IEEE paper (8 pages)	Done	report/main.tex, compiled on Overleaf
GitHub repository (private)	Done	Israfil0077/banglish-hate-speech-xai
Attention visualisation	Not done	Optional in the plan, no time
Rationale-supervised training	Future work	Needs far more rationales than our 100
LLM data augmentation for rare categories	Future work	Deferred by plan
Counter-speech suggestions (Claude API)	Future work	Deferred; needs a paid API key and evaluation
Large annotation (500 to 1000 comments, 3+ annotators)	Future work	Deferred by plan
Full fairness audit across communities	Future work	Only the small keyword test was done
Student IDs in the paper	Open	Author block still says "to be added"

3. Requirements (SRS)

The system must classify a Banglish comment, name its target categories and explain the decision in a few seconds, using only free tools (Google Colab and Drive).

Functional requirements

ID	Requirement	How it is met
FR1	Accept a comment in transliterated Bangla	Text box in the demo; any Latin-script text
FR2	Clean input the same way as training data	Remove URLs, collapse extra spaces; keep spelling, emoji, case
FR3	Predict hate or not hate with a probability	Binary head, sigmoid, threshold 0.58 tuned on validation

ID	Requirement	How it is met
FR4	Predict any number of the 8 categories	Category head, 8 sigmoids, one tuned threshold each; only shown if the comment is hate
FR5	Explain the decision word by word	SHAP (default) or LIME, words coloured red (towards hate) or blue (towards not hate)
FR6	Let the user switch explanation method	SHAP / LIME switch, re-explains without re-predicting
FR7	Offer ready examples and the known weakness	Example list plus a "Known weakness" list from the bias test
FR8	Save every figure, table and number for the report	results/ folder, registered in results/ASSETS.md
FR9	Survive Colab disconnects	Model saved to Drive after every epoch; training resumes

Non-functional requirements

ID	Requirement	Target / result
NFR1	Accuracy comparable to published work	On par with BanTH XLM-R (76.23 vs 77.35 binary macro-F1)
NFR2	Response time	Prediction plus explanation within a few seconds on a T4 GPU
NFR3	Reproducibility	Fixed seed 42, library versions saved, official splits
NFR4	Honesty	Every reported number comes from a saved file; no hand-typed results
NFR5	Privacy and ethics	Annotators anonymised as A and B; content warning; tool assists humans, never removes content alone
NFR6	Cost	Free: Colab T4, Google Drive, Gradio, open-source libraries
NFR7	Usability	One-click examples, readable on a projector and a phone, light and dark mode

Hardware and software

Part	What we used
Compute	Google Colab, Tesla T4 GPU (training and demo); CPU is enough for data notebooks
Storage	Shared Google Drive folder ML_Project (about 3 GB with model files)
Language and libraries	Python 3.13, PyTorch 2.11, Transformers 5.16, Datasets 4.8, scikit-learn 1.6, pandas, matplotlib, lime, shap 0.52, gradio 6.26, openpyxl
Annotation	Google Sheets editing Excel files in Drive
Paper	LaTeX (IEEEtran), compiled on Overleaf
Version control	Git + GitHub (private repository)

4. Data

We use BanTH, 37,350 YouTube comments in transliterated Bangla, with its official train / validation / test split of 29,879 / 3,736 / 3,735 comments. 28.4% of comments are hate in every split. We load it from Hugging Face (aplycaebous/BanTH) and never change the splits, so our numbers are comparable with the BanTH paper.

Columns: Text (the comment), Label (1 = hate, 0 = not hate), 8 category columns (0/1: Political, Religious, Gender, Personal Offense, Abusive/Violence, Origin, Body Shaming, Misc), plus bangla and english versions supplied with the dataset (kept for reference, never used for training; the English versions are sometimes wrong).

Label counts

Label	Train	Validation	Test
Non-hate	21,384	2,674	2,673
Hate	8,495	1,062	1,062
Personal Offense	5,752	710	715
Abusive/Violence	2,335	293	304
Political	978	111	117
Misc	207	25	30

Label	Train	Validation	Test
Gender	180	24	23
Origin	178	26	23
Religious	110	22	18
Body Shaming	92	17	14

The categories are very unbalanced: Body Shaming has only 14 test comments, so its scores are unreliable. A comment can carry several categories (1,165 training comments have 2, 80 have 3, 4 have 4).

Checks we ran (all passed)

- Split sizes equal the published ones; all 12 columns present; no missing values; all labels 0 or 1.
- Label and categories agree: 0 non-hate comments with a category, 0 hate comments without one.
- Duplicates: 8 repeated comments in train, 2 texts with conflicting labels, only 1 test comment also in train (no real leakage).

Cleaning (deliberately light)

Step	Kept or removed	Why
URLs	Removed (3 comments had one)	A link carries no language signal
Extra spaces and line breaks	Collapsed to one space	Same text however it was typed
Spelling variation	Kept	It is the real difficulty of Banglish; fixing it would make the task artificially easy
Emoji and punctuation	Kept	They carry tone (for example angry faces)
Upper / lower case	Kept	The tokenizer handles case
Stop words	Kept, no stemming	Transformers read full sentences; explanations must show the words people wrote
Rows	None removed	Keeps official split sizes

Dataset version note: the BanTH paper describes 7 categories and lists 10,282 hate comments, but the public release has 8 categories (adds Misc) and 10,619 hate comments; for example Body Shaming is 68 in the paper's train split and 92 in ours. Split sizes match. So the published baselines may use a slightly different label version, and small gaps should not be over-read.

5. Model

We fine-tune XLM-RoBERTa base (278M parameters) with two output heads, one for hate or not and one for the 8 categories, trained together in 18.4 minutes on a T4 GPU.

Why XLM-R

- In the BanTH paper it was the best normally fine-tuned model (binary macro-F1 77.35), only 0.01 behind the best overall model (TB-mBERT), which needed extra pre-training we could not repeat.
- It was pre-trained on 100 languages, including Bangla and English, so its sub-word vocabulary covers Latin-script Bangla and English words.
- The base size fits and trains fast on a free T4 GPU; the large size would not.
- BanglaBERT was trained on Bangla script, which our Banglish text does not use.

How it works, step by step

1. **Tokenizer:** splits the comment into sub-word pieces. An unusual spelling becomes several known pieces instead of an unknown word, which is why spelling variation is not fatal. Median comment = 15 tokens; we cut at 128 tokens, which fits 99% of comments (only 0.6% are shortened).
2. **Encoder (12 transformer layers):** each layer uses self-attention, so every token looks at every other token and updates its meaning from context. This is how *baccha* can mean "child" in one sentence and be part of an insult in another.
3. **Summary vector:** we take the output of the first token as a summary of the whole comment (768 numbers).
4. **Two heads (small linear layers):**
 - Binary head: 1 output, passed through a sigmoid to get $P(\text{hate})$ between 0 and 1.
 - Category head: 8 outputs, each with its own sigmoid, because a comment can have several categories at once (softmax would force exactly one).
5. **Decision:** a comment is hate if $P(\text{hate})$ is at least 0.58. Categories are only reported for comments predicted as hate, like the two-stage setup in the BanTH paper.

The sigmoid turns any score z into a probability:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

Loss (what training minimises)

Both heads use binary cross-entropy (BCE), and the total loss is their sum:

$$\mathcal{L} = \mathrm{BCE}(p_b, y_b) + \frac{1}{8} \sum_{c=1}^8 \mathrm{BCE}_{w_c}(p_c, y_c), \quad \mathrm{BCE}(p, y) = -\left[y \log p + (1-y) \log(1-p) \right]$$

Rare categories get a larger weight w_c on their positive examples, so the model does not learn to always say "no" for them:

$$w_c = \sqrt{\frac{N^{-}_c}{N^{+}_c}}$$

Worked example: Body Shaming has 92 positive and 29,787 negative training comments, so $w = \sqrt{(29,787 / 92)} = \sqrt{323.8} \approx 18.0$. Personal Offense has 5,752 positives, so $w = \sqrt{(24,127 / 5,752)} \approx 2.05$. We take the square root so the rare classes help without taking over.

Training setup

Setting	Value	Why
Optimizer	AdamW, learning rate 2e-5, weight decay 0.01	Same learning rate as the BanTH paper
Schedule	Linear warm-up for 10% of steps, then linear decay to 0	Standard for fine-tuning transformers
Batch size	32	Same as BanTH
Epochs	Up to 5, early stopping after 2 epochs without improvement in validation loss	Same rule as BanTH; best epoch was 4
Max length	128 tokens	Measured: covers 99% of comments, much faster than BanTH's 512
Dropout, gradient clipping	0.1, 1.0	Stability
Precision	Mixed fp16	About twice as fast on a T4
Seed	42	Repeatable runs
Checkpoints	Saved to Drive every epoch (last_model.pt) and when validation loss improves (best_model.pt)	Survives Colab disconnects

Decision thresholds. Instead of 0.5 for everything, we try thresholds from 0.05 to 0.95 on the validation set and keep the one with the best F1, separately for the binary output (0.58) and each category (Political 0.75, Religious 0.44, Gender 0.71, Personal Offense 0.62, Abusive/Violence 0.68, Origin 0.84, Body Shaming 0.63, Misc 0.31). The test set is never used for tuning.

6. Explanations: LIME and SHAP

Both methods give every word a score: positive pushes the comment towards hate, negative towards not hate. A "word" is a piece of text between spaces, the same unit the human annotators marked, so the two can be compared word by word. Both explain the binary $P(\text{hate})$.

LIME (Local Interpretable Model-agnostic Explanations)

1. Make 1,000 versions of the comment with random words deleted.

2. Ask the model for $P(\text{hate})$ of each version.
3. Fit a simple weighted linear model: $P(\text{hate}) \approx w_0 + w_1 \cdot (\text{word 1 present}) + w_2 \cdot (\text{word 2 present}) + \dots$ Versions closer to the original get more weight.
4. The weight w of each word is its LIME score.

SHAP (SHapley Additive exPlanations)

SHAP comes from game theory: the words are "players" who together produce the prediction, and each gets its fair share. A word's Shapley value is its average extra contribution over all possible orders in which words could be added:

$$\phi_i = \sum_{S \subseteq F, i \in S} \frac{(|S|-1)! (|F|-|S|)!}{|F|!} (f(S \cup \{i\}) - f(S))$$

Here F is the set of all words, S is a subset of words kept, and $f(S)$ is $P(\text{hate})$ when only the words in S are kept. Trying every subset is too slow, so we use SHAP's partition explainer with 500 model calls per comment. Removed words are deleted, as in LIME.

The key property: the scores add up exactly to the prediction.

$$f(x) = \phi_0 + \sum_i \phi_i$$

Worked example (test_03181, a real hate comment, $P(\text{hate}) = 0.981$)

Word	LIME score	SHAP score
Khankir	+0.090	+0.157
bacca	+0.068	+0.133
Baler	+0.063	+0.130
marte	+0.013	+0.100
polar	+0.024	+0.065
ekta	−0.001	+0.065
giri	+0.016	+0.041
kaner	+0.001	−0.009
police	+0.003	−0.009
dekhaitam.	+0.019	−0.030
parle	+0.015	−0.037
niche	−0.007	−0.052

SHAP check: the empty comment scores $\phi_0 = 0.426$, the 12 word scores add up to 0.555, and $0.426 + 0.555 = 0.981$, exactly the model's output. Both methods rank the same three insult words first (Khankir, bacca, Baler).

Where they disagree. On test_00575, a normal comment wrongly flagged ($P = 0.63$), both blame *Tor* and *Tore* (the informal "you"), but some words get opposite signs: *porone* is +0.005 in LIME and −0.132 in SHAP. This is why we do not trust explanations blindly and test them against people (section 9).

Settings: LIME 1,000 samples, SHAP partition explainer 500 evaluations, both with seed 42, lime and shap Python libraries. Across 8 fixed example comments, LIME and SHAP share on average 0.71 of their top-3 hate words.

7. Metrics and calculations

We use the same metrics as the BanTH paper for classification, HateXplain's metrics for plausibility and ERASER's for faithfulness. Every worked number below comes from our real test results.

Confusion matrix (test set, threshold 0.58)

	Predicted non-hate	Predicted hate
True non-hate (2,673)	TN = 2,331	FP = 342
True hate (1,062)	FN = 374	TP = 688

$$\text{Precision} = \frac{TP}{TP+FP}, \quad \text{Recall} = \frac{TP}{TP+FN}, \\ F_1 = \frac{2 \cdot P \cdot R}{P + R}$$

Worked example:

1. Hate class: $P = 688 / 1,030 = 0.668$, $R = 688 / 1,062 = 0.648$, $F1 = 0.658$.
2. Non-hate class: $P = 2,331 / 2,705 = 0.862$, $R = 2,331 / 2,673 = 0.872$, $F1 = 0.867$.
3. **Binary macro-F1** = average of the two F1 values = $(0.658 + 0.867) / 2 = \mathbf{0.762 (76.23\%)}$.
Macro means each class counts equally, which is fair when hate is only 28%.
4. **Accuracy** = $(2,331 + 688) / 3,735 = \mathbf{80.83\%}$.
5. **Macro-accuracy** (balanced accuracy, what BanTH calls Acc.) = average recall of the two classes = $(0.872 + 0.648) / 2 = \mathbf{75.99\%}$.

Multi-label metrics (8 categories)

- **Macro-F1:** F1 computed for each category separately, then averaged. A category with $F1 = 0$ pulls the average down hard, which is why ours is low.
- **Subset accuracy:** share of comments whose full set of 8 yes/no labels is exactly right.
- **Hamming loss:** share of individual labels that are wrong (lower is better).

$$\text{Hamming loss} = \frac{1}{N \cdot 8} \sum_{n=1}^N \sum_{c=1}^8 \mathbb{1}[\hat{y}_{nc} \neq y_{nc}]$$

Toy example (made up for illustration): a comment is truly {Political, Personal Offense}; the model says {Personal Offense}. Subset accuracy for it = 0 (not an exact match), Hamming loss = 1 wrong out of 8 = 0.125.

Cohen's Kappa (annotator agreement)

$$\kappa = \frac{p_o - p_e}{1 - p_e}$$

p_o is how often the two annotators made the same mark/no-mark decision, and p_e is how often they would agree by pure chance. Our numbers over 551 words: $p_o = 0.784$. Annotator A marked about 45% of words and B about 39%, so $p_e = 0.45 \times 0.39 + 0.55 \times 0.61 \approx 0.51$. Then $\kappa = (0.784 - 0.51) / (1 - 0.51) \approx \mathbf{0.559}$, "moderate" agreement (0.41 to 0.60 on the Landis and Koch scale).

Plausibility (do the explanation words match the human words?)

- **AUPRC:** how well the word scores rank the human-marked words above the others (1 = perfect).
- **Token F1:** take as many top-scored words as the human marked (k), then F1 between those k words and the human words.
- **IOU F1:** share of comments where overlap / union of the two word sets is at least 0.5.

Example: human marks {ghuira, ghuira, mara, kha} ($k = 4$); SHAP's top 4 = {tui, ghuira, ghuira, mara}. Overlap = 3, so token F1 = $2 \times 3 / (4 + 4) = 0.75$ and IOU = $3 / 5 = 0.6$, which counts as a hit.

Faithfulness (does the model really use those words?)

$$\text{Comprehensiveness} = p(x) - p(x \setminus r), \quad \text{Sufficiency} = p(x) - p(r)$$

x is the comment, r the top words, and p is $P(\text{hate})$. Comprehensiveness asks "does $P(\text{hate})$ drop when I delete these words?" (higher is better). Sufficiency asks "are these words enough on their own?" (lower is better; negative means the words alone look even more hateful). We use k = the number of human-marked words, and also AOPC: the average over deleting the top 10%, 20%, 30% and 50% of words. A **random** word order is the baseline every method must beat.

False positive rate (bias test): $\text{FPR} = \text{flagged harmless sentences} / \text{all harmless sentences}$. Because every probe sentence is harmless, every flag is a mistake.

8. Results

Our model is on par with the published fine-tuned baselines, not better: binary macro-F1 76.23 against 77.35 for the BanTH paper's XLM-R, and category macro-F1 28.89 on hate comments against 29.29.

Comparison with the BanTH paper (test set, %)

Model	Binary macro-F1	Binary macro-acc.	Multi-label macro-F1	Subset acc.	Hamming loss
BanglaBERT (paper)	76.50	81.04	20.82	54.08	7.26
mBERT (paper)	74.97	80.37	28.24	52.19	7.78
XLM-R (paper)	77.35	81.37	29.29	53.28	7.23
TB-BERT (paper, extra pre-training)	76.27	79.25	30.17	54.19	7.18
TB-mBERT (paper, best binary)	77.36	82.57	27.07	54.71	7.28

Model	Binary macro-F1	Binary macro-acc.	Multi-label macro-F1	Subset acc.	Hamming loss
GPT-4o few-shot (paper, best multi-label)	68.77	74.18	39.53	26.76	14.16
Ours, multi-label on all test comments	76.23	75.99	19.53	72.37	4.34
Ours, multi-label on hate comments only	76.23	75.99	28.89	31.45	11.03

[BanTH paper, Table 7](#), numbers checked against [arXiv 2410.13281v4](#).

Why the comparison is not exact

- The paper says "Acc." is macro-accuracy, but its values are close to our plain accuracy (80.83), not our macro-accuracy (75.99). We report both.
- The paper does not say which comments its multi-label scores use, so we report two versions; its values fall between them.
- The public dataset differs from the paper's label version (section 4).

Threshold effect (binary)

Threshold	Macro-F1	Macro-acc.	Accuracy	Hate precision	Hate recall
0.5	75.50	76.20	79.54	62.89	68.46
0.58 (tuned on validation)	76.23	75.99	80.83	66.80	64.78

Per category (test set, tuned thresholds, %)

Category	Train examples	Test examples	Precision	Recall	F1 (all comments)	F1 (hate comments)
Personal Offense	5,752	715	51.5	62.0	56.2	67.1
Abusive/Violence	2,335	304	50.8	43.1	46.6	50.1
Political	978	117	38.6	33.3	35.8	50.7
Gender	180	23	8.6	13.0	10.3	23.5
Misc	207	30	8.0	6.7	7.3	8.2
Religious	110	18	0.0	0.0	0.0	19.0
Origin	178	23	0.0	0.0	0.0	0.0
Body Shaming	92	14	0.0	0.0	0.0	12.5

The pattern is clear: the more training examples, the better the category. Religious, Origin and Body Shaming fail on all comments, because their thresholds were tuned on only 17 to 26 validation examples and do not carry over to the test set.

Training curve: validation loss was lowest at epoch 4 of 5 (train loss kept falling, validation loss rose slightly at epoch 5), so `best_model.pt` is from epoch 4. Validation category macro-F1 was still rising (10.0, 14.5, 18.0, 21.9, 24.3 over the 5 epochs), so longer training might help the categories.

Error pattern: 35.2% of hate comments are missed and 12.8% of non-hate comments are wrongly flagged. The explanations show that many false alarms are driven by informal pronouns such as *tui*, *tor*, *tore*, which appear in insults but also between friends.

9. Human rationale check

SHAP and LIME pick the same words people see as hateful far more often than chance (SHAP token F1 0.715 vs 0.518 for random words), and the model really depends on those words: deleting SHAP's top words lowers $P(\text{hate})$ by 0.48 on average, against 0.16 for random words.

How the rationale set was made

1. **Sample:** 100 hate comments from the test set (the model never trained on them), 3 to 30 words long, picked with seed 42 and a quota per category so rare categories appear (Body Shaming 8, Religious 10, Origin 10, Gender 12, Misc 10, Political 19, Abusive/Violence 24, Personal Offense 45; one comment can have several).
2. **Sheets:** one Excel file per annotator in Drive, edited in Google Sheets. Each comment is split one word per cell; the annotator types 1 under every word that makes it hateful.
3. **Guidelines (HateXplain idea):** mark only the words that are the reason; mark every word of an insulting phrase (for example both words of *kuttar baccha*); do not mark names unless used as an insult; punctuation and emoji stuck to a word count with it; "Nothing hateful" column if nothing is hateful; work alone without seeing the other sheet.
4. **No AI suggestions** were shown, so every mark is human.
5. **Annotators:** two team members, reported only as Annotator A and Annotator B.
6. **Coverage:** A annotated 93 comments, B 57, and 55 were annotated by both. Blank rows mean "not done" and are left out, never counted as "no hateful words".

Agreement: Cohen's Kappa = 0.559 over 551 words in the 55 shared comments (moderate), raw agreement 78.4%. A marks about 4.53 words per comment and B 3.93, so A marks slightly more.

Plausibility (against Annotator A, 91 comments with marked words)

Method	AUPRC	Token F1	IOU F1
SHAP	0.808	0.715	0.604
LIME	0.805	0.683	0.527
Random	0.627	0.518	0.352

Against Annotator B (55 comments) the order is the same: SHAP 0.690, LIME 0.658, random 0.466 token F1. So the result does not depend on who annotated.

Faithfulness (91 comments, k = number of human-marked words)

Rationale	Comprehensiveness (higher is better)	Sufficiency (lower is better)	Comprehensiveness, AOPC	Sufficiency, AOPC
Human words	0.41	-0.02	-	-
LIME	0.47	-0.13	0.41	-0.12
SHAP	0.48	-0.13	0.42	-0.13
Random	0.16	0.18	0.08	0.27

What this means

- The explanations are faithful: the words they pick really drive the prediction. This differs from HateBRXplain (Portuguese), where explanations looked plausible but were not faithful; one possible reason is that hate in BanTH is often carried by explicit insult words.
- Deleting the words people marked also drops P(hate) by 0.41, so the model mostly relies on the same words humans consider hateful.
- The explanations still show a bias: in one example both annotators marked *ghuira* *ghuira mara kha*, but LIME and SHAP ranked the pronoun *tui* above *kha*. This led to the keyword bias test (section 10).

10. Keyword bias test and the fix

The model wrongly flags 26.1% of harmless sentences that contain insult-like words, against 3.2% of harmless sentences without them. Oversampling harmless training comments with those words cut false alarms on real test comments from 74.8% to 33.8%,

but lowered category macro-F1 from 28.89 to 22.05, so the main model stays our main result.

How we found it. In the demo, the harmless sentence *amar kukurer baccha hoyeche* ("my dog had puppies") was flagged as hate with $P = 0.97$, and SHAP and LIME blamed *kukur* and *baccha*. Earlier examples showed the same with *tui* / *tor* / *tore*. We tested whether this is a pattern.

Test design

- Group A: 69 everyday harmless Banglish sentences containing a risky word (30 words, for example *kukur*, *kuttar*, *baccha*, *tui*, *pagol*, *mohila*, *meye*, *hindu*, *muslim*, *india*, *awami*, *bnp*, *mota*, *kalo*, *hijra*), checked by the team as natural and non-hateful.
- Group B: 31 harmless sentences with no risky word (control).
- Every flag is a false positive, because all 100 sentences are harmless. We also ran SHAP on each sentence to see whether the risky word was the main reason.

Results

Group	Sentences	Flagged as hate	False positive rate	Mean P(hate)	Risky word is SHAP's top word
A: with risky word	69	18	26.1%	0.33	55.1%
B: without	31	1	3.2%	0.12	–

Why it happens (from the training data). The words the model over-flags almost always appear in hate comments in BanTH: *kuttar* 94% hate, *kukur* 82%, *baccha* 77%, *pagol* 59%, against 28.4% overall. The model learned "this word means hate". Group words such as *hindu* (30%) and *bnp* (24%) are not tied to hate in the data, and harmless sentences with them were never flagged. So the bias is about insult-like and some gender and appearance words (all 3 *mohila* sentences were flagged), not about naming a religion or party.

Does BanTH have it too? Probably any model trained on BanTH learns the same shortcut, because the cause is in the data, but the BanTH paper never tested harmless sentences, so we cannot claim numbers for their models.

The fix: oversampling (notebook 10)

1. From the training data only, list words that appear in at least 20 comments with a hate rate of at least 60%: 99 words.
2. Find the 1,307 non-hate training comments that contain one of them.
3. Repeat each 3 extra times, so the model sees these words used harmlessly more often (29,879 to 33,800 training rows).

- Retrain with exactly the same settings and seed. The probe sentences are never used for training or choosing words.

Model	Binary macro-F1	Category macro-F1 (hate)	Probe FPR, risky word	Probe FPR, no risky word	Real test non-hate with listed word (151)	Real test non-hate without (2,522)
Main model	76.23	28.89	26.1%	3.2%	74.8%	9.1%
With oversampling	75.46	22.05	18.8%	0.0%	33.8%	9.2%

The fix more than halves false alarms on real comments with these words, without changing false alarms elsewhere, but category macro-F1 drops by 6.84 points. The fixed model early-stopped at epoch 2, so part of that drop may come from shorter training. We keep the main model in the paper and the app, and report the fix as a trade-off. *mohila*, *meve*, *kalo* and *mota* did not pass the 60% rule, so the fix does not target them.

11. Demo app

The demo is a Gradio web app started from notebook 08 in Colab; it gives a public link that works on any device while the notebook runs.

What it shows

- Verdict card:** "Hate speech" (red) or "Not hate speech" (green), P(hate) as a percentage, and a bar with the 0.58 threshold marked.
- Why this decision?:** every word coloured red (towards hate) or blue (towards not hate), darker = stronger; hover a word for its exact score; the top 3 hate words listed.
- SHAP / LIME switch:** re-explains the same comment with the other method.
- Hate categories:** 8 bars, the detected ones in blue with a "detected" tag (only for comments predicted as hate).
- Try an example:** 9 one-click comments (the 8 fixed test comments from notebook 05 plus a clearly positive one).
- Known weakness box:** a short note with the bias test result (26% vs 3%) and 6 harmless sentences the model wrongly flags, picked by a rule (the most confidently flagged sentence per risky word).
- How it works:** model, data and key results, read from the saved metrics files so they always match the report; a note that it is a research prototype that helps moderators and never removes content on its own.

How to start it (each time)

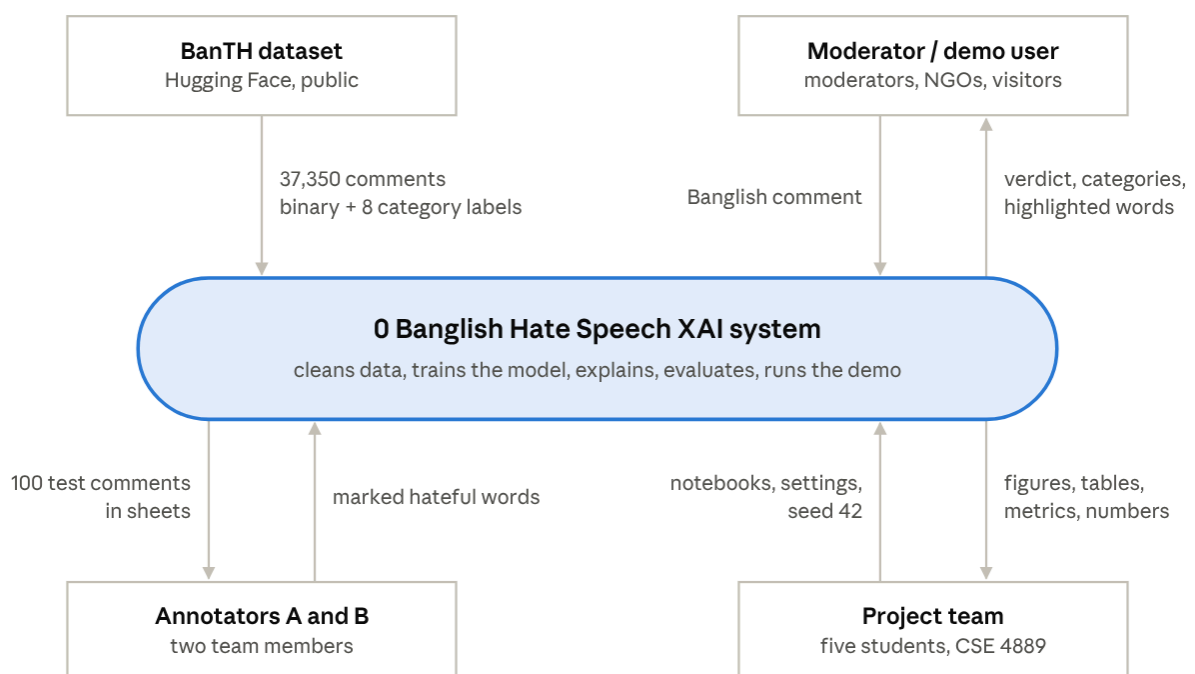
- Open notebooks/08_demo_app.ipynb from Drive in Colab.

2. Runtime, Change runtime type, T4 GPU.
3. Runtime, Run all; allow Drive access.
4. After about 2 to 3 minutes the last cell prints a link ending in gradio.live. Open it (add ? __theme=light for a white page, ? __theme=dark for dark).
5. Click one example once to warm up the model before presenting.

Keep in mind: a new link is made at every launch; the link works only while the Colab tab runs (Colab disconnects after about 90 minutes idle) and expires after 72 hours. Backup: screenshots in results/figures/fig_09_demo_*.png.

12. Context diagram (DFD level 0)

Four outside parties exchange data with the system

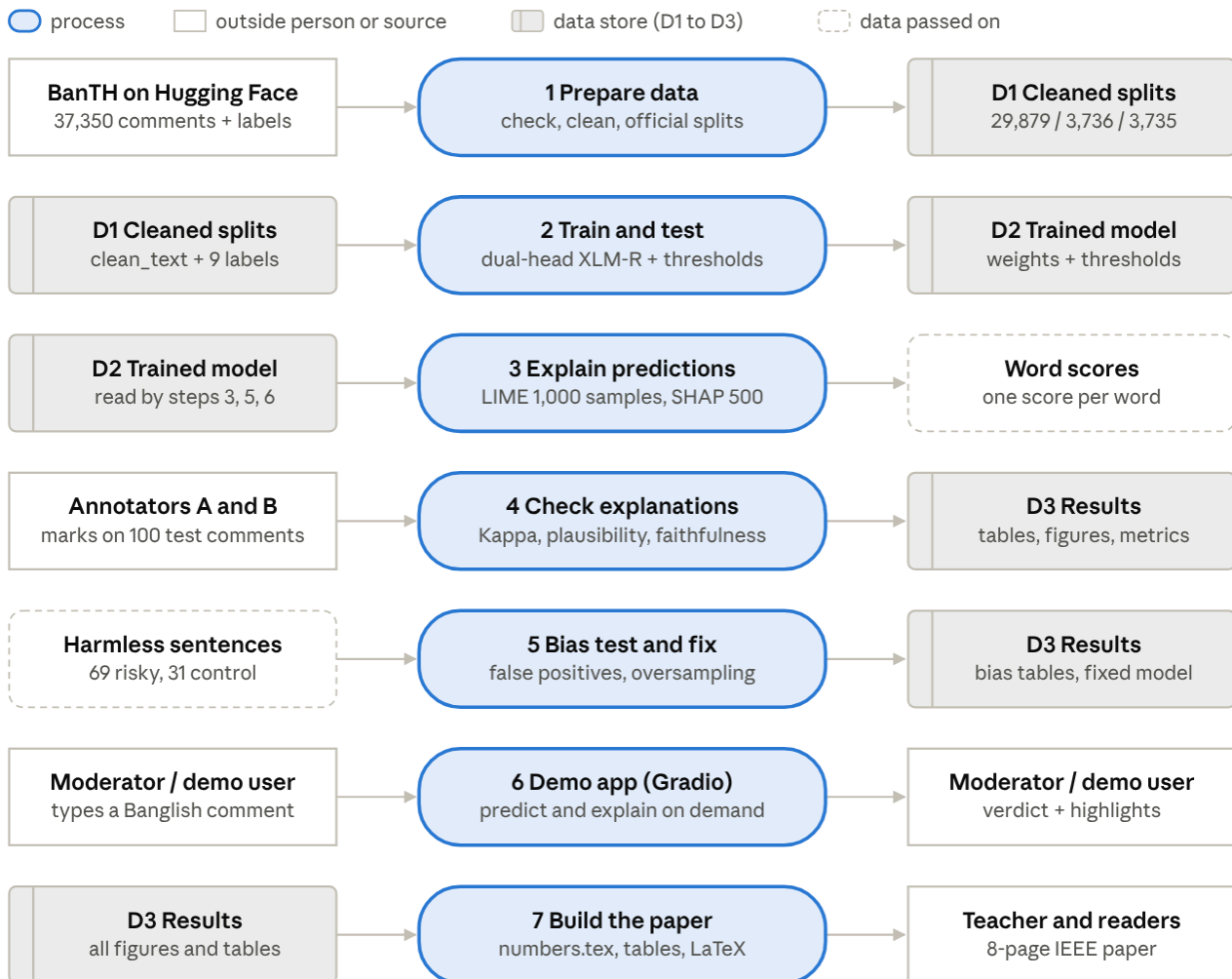


context diagram (DFD level 0) · 1 system, 4 outside parties

The whole system is one process (0). Data comes in from BanTH and from the annotators, the moderator sends a comment and gets back a verdict with highlighted words, and the team runs the notebooks and gets every result saved for the paper.

13. Data flow diagram (DFD level 1)

Seven processes turn BanTH into results, a demo and a paper



data flow diagram (DFD level 1) · 7 processes, 3 data stores

Each row is one process with the data it reads (left) and writes (right). Three stores hold everything between steps: D1 the cleaned splits, D2 the trained model and thresholds (read by explanation, bias test and demo), and D3 the results folder the paper is built from.

14. Project files and how to reproduce

Everything lives in the shared Drive folder ML_Project (Colab path /content/drive/MyDrive/ML_Project) and, without the model files and BanTH data, in the private GitHub repository [Israfil0077/banglish-hate-speech-xai](https://github.com/Israfil0077/banglish-hate-speech-xai).

Notebooks, in the order to run them

Notebook	What it does	Runtime	Main outputs
00_setup_drive_folders	Creates the folders, records library versions, starts ASSETS.md	CPU, 1 min	folder tree figure, environment.txt
01_data_exploration	Loads BanTH, checks, cleans, saves splits	CPU, 1 min	tables T01 to T06, figures F02, F03, data/processed
03_baseline_table	Saves the BanTH paper's results as baseline	CPU, seconds	T07 to T09, metrics_baseline.json
04_model_training	Trains the dual-head XLM-R, tunes thresholds, tests	T4, about 20 min	best_model.pt, model_config.json, T10 to T15, F04 to F06
05_explainability	LIME and SHAP on 8 fixed test comments	T4, 5 min	F07.1 to F07.8, T16
06_rationale_sheet	Picks 100 comments, makes the annotation sheets	CPU, 1 min	two Excel sheets, T17
07_rationale_evaluation	Kappa, plausibility, faithfulness	T4, 10 min	T18 to T20, F08, banth_xplain_mini.csv
08_demo_app	Gradio web app with public link	T4	the app
09_keyword_bias	Harmless-sentence bias test	T4, 3 min	T21 to T23, F10
10_debias_training	Oversampling fix and comparison	T4, about 25 min	T24, T25, F11

There is no notebook 02: the number was reserved for annotation prep in the original plan, and that work became notebook 06.

Folders

Folder	Contents
src/model.py	Dual-head model class and a loader for the trained weights

Folder	Contents
src/explainability/explain.py	Shared LIME and SHAP functions (used by notebooks 05, 07, 08, 09)
data/raw, data/processed	BanTH splits as downloaded and cleaned (not on GitHub)
data/banth_xplain	Rationale sample, guidelines, the annotated mini dataset (sheets not on GitHub)
data/bias_probe	The 100 harmless probe sentences
results/figures, results/tables	Every figure (300 dpi PNG) and table (CSV + LaTeX)
results/evaluation_reports	All metrics as JSON, training logs, predictions, environment files
results/model_checkpoints	Model weights, about 1.1 GB each (Drive only)
results/ASSETS.md	List of every saved figure and table with its caption and report section
report	IEEE paper: main.tex, references.bib, numbers.tex, tables, figures, build_report_assets.py

Rules we followed

- Every notebook starts by mounting Drive, uses seed 42, explains each code cell in a text cell before it, and ends with a "Remember this" summary.
- Every figure, table and number is saved to results/ and listed in ASSETS.md.
- report/build_report_assets.py reads results/ and writes numbers.tex and the LaTeX tables, so no number in the paper is typed by hand.

Update the paper after a change: run build_report_assets.py, upload main.tex, numbers.tex and tables/ to Overleaf, recompile.

Update GitHub after a change: git add ., git commit -m "what changed", git push (from the ML_Project folder).

15. Limitations, what we could not do, and future work

The biggest limits are the small rationale set (100 comments, 2 annotators), weak rare categories, and a keyword bias we could reduce only at a cost.

Limitations

- **Small rationale set:** 100 comments; only 55 annotated by both people; agreement is moderate (Kappa 0.559); the main plausibility reference is one annotator.
- **Rare categories:** Religious, Origin and Body Shaming get F1 0 on all test comments; their test scores rest on 14 to 23 comments.
- **Baseline comparison is approximate:** different label version in the public dataset, unclear macro-accuracy definition, unclear multi-label scope in the BanTH paper.
- **Not better than the baselines:** we are on par, 1.12 points below XLM-R on binary macro-F1; the best published models used extra pre-training on transliterated text that we did not repeat.
- **Keyword bias:** 26.1% of harmless sentences with insult-like words are flagged; the fix lowers category F1.
- **Bias test size:** 100 sentences show a pattern, not an exact rate.
- **Emoji** cannot be drawn in our figures, although the model reads them.
- **Single training run** with seed 42; no averaging over several seeds.
- **The app** depends on Colab: links expire and sessions disconnect.

Planned but not done (future work)

Item	Why not now	What it would take
Rationale-supervised training	Needs far more rationales	Extra loss that pushes the model's attributions towards the human words (HateXplain showed this reduces bias)
Counter-speech suggestions	Needs a paid Anthropic API key and a way to evaluate replies	For flagged comments, draft a calm reply a moderator can review, rated by moderators
LLM augmentation for rare categories	Deferred by plan	Generate and human-review extra Body Shaming and Misc examples
Large annotation	Time	500 to 1,000 comments, 3 or more annotators, public release as a rationale dataset
Full fairness audit	Time	False positive rates per target community on a larger probe set
Attention visualisation	Optional, no time	Show attention weights next to LIME and SHAP

Item	Why not now	What it would take
Better bias fix	Current fix costs 6.84 category F1 points	Longer training for the fixed model, counterfactual examples, or rationale supervision
Stronger encoder	Not available in time	Try encoders further pre-trained on transliterated Bangla (best in the BanTH paper)

Ethics

- BanTH is public YouTube comments; we show offensive examples only to explain the task and never share user identities.
- Annotators saw a content warning and were told to take breaks.
- Annotators are anonymised as A and B in all outputs.
- The tool must assist people, not replace them: its false alarms on everyday words could silence ordinary users if used without review.

16. Glossary and sources

Glossary

Term	Meaning
Banglish / transliterated Bangla	Bangla written with English letters
Binary classification	Two answers: hate or not hate
Multi-label classification	Any number of categories can apply to one comment
Transformer / encoder	A neural network that reads all words together and uses context (self-attention)
XLM-R (XLM-RoBERTa)	A multilingual transformer pre-trained on 100 languages
Fine-tuning	Training a pre-trained model a bit more on our own labelled data
Token	A sub-word piece the model reads
Sigmoid	Turns a score into a probability between 0 and 1

Term	Meaning
BCE (binary cross-entropy)	The loss for yes/no predictions
Threshold	The probability above which we say "yes" (0.58 for hate)
Macro-F1	F1 averaged over classes, each class counted equally
Subset accuracy / Hamming loss	Exact-match rate of all labels / share of wrong labels
XAI	Explainable AI: showing why a model decided
LIME / SHAP	Methods that score each word's effect on the prediction
Rationale	The words a person marks as the reason a comment is hateful
Cohen's Kappa	Agreement between two annotators, corrected for chance
Plausibility	Do explanation words match human words?
Faithfulness	Does the model really depend on the explanation words?
Comprehensiveness / sufficiency	P(hate) drop when the top words are deleted / when only they are kept
False positive rate	Share of harmless texts wrongly flagged as hate
Oversampling	Showing some training examples more often
Gradio	Python library for simple web apps
Colab T4	Free Google GPU used for training and the demo

Sources

- Haider et al., BanTH, Findings of NAACL 2025: [ACL Anthology](#), [arXiv](#), [dataset](#)
- Mathew et al., HateXplain, AAAI 2021: [paper](#)
- Salles et al., HateBRXplain, COLING 2025: [ACL Anthology](#)
- Lee et al., K-MHaS, COLING 2022: [ACL Anthology](#)
- Romim et al., BD-SHS, LREC 2022: [ACL Anthology](#)
- Hasan et al., LLM-Based Multi-Task Bangla Hate Speech Detection, ACL 2026: [ACL Anthology](#)

- Trager et al., MFTCXplain, Findings of EMNLP 2025: [ACL Anthology](#)
- Conneau et al., XLM-R, ACL 2020: [ACL Anthology](#)
- DeYoung et al., ERASER, ACL 2020: [ACL Anthology](#)
- Ribeiro et al., LIME, KDD 2016: [doi](#)
- Lundberg and Lee, SHAP, NeurIPS 2017: [paper](#)
- Yadav et al., user-based evaluation of explainability for misogyny, Applied AI Letters 2026: [doi](#); explainable multimodal misogyny app: [arXiv](#)
- Full, verified list with all 22 references: report/references.bib